

10

Decision Trees

Decision trees are very different representations of decision functions, with different advantages and disadvantages than those of the previous sections.

Classification Trees with Discrete Features

When a decision tree is used for classification it is often called a **classification tree**. The most straight-forward situation is when the features are also discrete-valued. As a concrete example, consider a decision tree for the task of finding Sibonelo (from Chapter 3). We wish to predict where Sibonelo will be (grass or lake) from three features: x_1 (weather: overcast, clear, or rainy), x_2 (time: day or night), and x_3 (moon phase: full or new).

Our hypothesis space is the set of all decision trees. A **decision tree** is graphical representation of a function. It is a directed rooted tree where each internal **node** node is associated with one binary test of one feature, and the **leaves** are associated with a prediction for the target, y . To evaluate the decision tree on a particular example, we start at the root (top) and proceed down a single path to a leaf. Each internal node is a binary test of one feature. Based on the outcome of the test on the example, we proceed down one of the branches. We follow this procedure recursively until we reach a leaf where the value of the decision tree for this input is given.

Figure 10.1 shows one possible classification tree, graphically. It predicts that Sibonelo is in the grass if it is not day and not clear. Otherwise, it predicts that Sibonelo is at the lake. This decision tree could also be written mathematically as

$$f(\vec{x}) = \begin{cases} \text{lake} & \text{if } x_2 = \text{day} \\ \begin{cases} \text{lake} & \text{if } x_1 = \text{clear} \\ \text{grass} & \text{otherwise} \end{cases} & \text{otherwise} \end{cases} \quad (10.1)$$

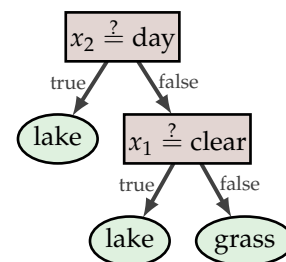


Figure 10.1: Example classification tree

$f(\vec{x})$ is the following as pseudo-code.

```

if  $x_2 = \text{day}$  then
  return lake
else
  if  $x_1 = \text{clear}$  then
    return lake
  else
    return grass
  
```

but such a representation quickly becomes unwieldy as the rule becomes more complicated.

Algorithm 10.2 is the algorithm to evaluate a decision tree. The tree, $\mathcal{T}$, is equal to $\text{leaf}(y)$ when it is just a leaf that evaluates to y . It is equal to $\text{node}(r, \mathcal{T}^{\text{true}}, \mathcal{T}^{\text{false}})$ when it is not a leaf: r is the test of the root node; $\mathcal{T}^{\text{true}}$ is the subtree for when $\tau(\vec{x})$ is true; and $\mathcal{T}^{\text{false}}$ is the subtree for when $\tau(\vec{x})$ is false.

Learning Classification Trees

There are *many* possible classification trees. If there are m features and each is binary, a full classification tree would have m levels and therefore 2^m leaves, each representing the output corresponding to a different possible input. If the problem had C classes, each of these leaves could take on a different one of the C possible values and thus there are (at least) C^{2^m} possible classification trees.

Therefore, we cannot search all possible classification trees to find the one that best fits the training data; there are simply too many. Instead, we will use a greedy algorithm that will “grow” the tree, downward from the top. We begin by selecting a node for the root of the tree. Consider the same example training data from Table 3.2. Figure 10.3 shows *two* (of the seven possible) examples of attribute tests that might be used for the root. For each, note that the training data are split by the choice of the root node: some go “left” (the true branch) and some go “right” (the false branch).

Our learning algorithm will consider all possible tests for this root node. It will pick one of these attribute tests (by a method we will discuss later). At that point, it will recurse down each branch of the tree.

For instance, if the learning algorithm chooses $x_1 \stackrel{?}{=} \text{overcast}$ as the root node, there are a few choices for the “true” branch. Two are shown in Figure 10.4.

Considering the two options shown in Figure 10.4, the one on the right ($x_3 \stackrel{?}{=} \text{full}$) is better: The resulting two branches have **pure** labels: Both the training examples in the left branch have the label “grass,” and all of the training examples (only one) in the right branch have the label “lake.” So, at this point, we do not need to recurse further. We can replace these branches with leafs and backtrack to the right branch of the root node to continue the process.

Scoring functions

In Figure 10.4 it is clear which node to add because one of the options perfectly splits the remaining training data. But, in Figure 10.3, it is less clear. Neither option perfectly separates the two classes. We

DT EVAL

In: $\mathcal{T}, \vec{x}$

Out: $f_{\mathcal{T}}(\vec{x})$

if $\mathcal{T} = \text{leaf}(y)$ **then**

return y

else

$\text{node}(r, \mathcal{T}^{\text{true}}, \mathcal{T}^{\text{false}}) \leftarrow \mathcal{T}$

if $r(\vec{x})$ **then**

return DT EVAL($\mathcal{T}^{\text{true}}, \vec{x}$)

else

return DT EVAL($\mathcal{T}^{\text{false}}, \vec{x}$)

Algorithm 10.2: Decision Tree Evaluation Algorithm. $\mathcal{T}$ is the tree representing the function $f_{\mathcal{T}}$.

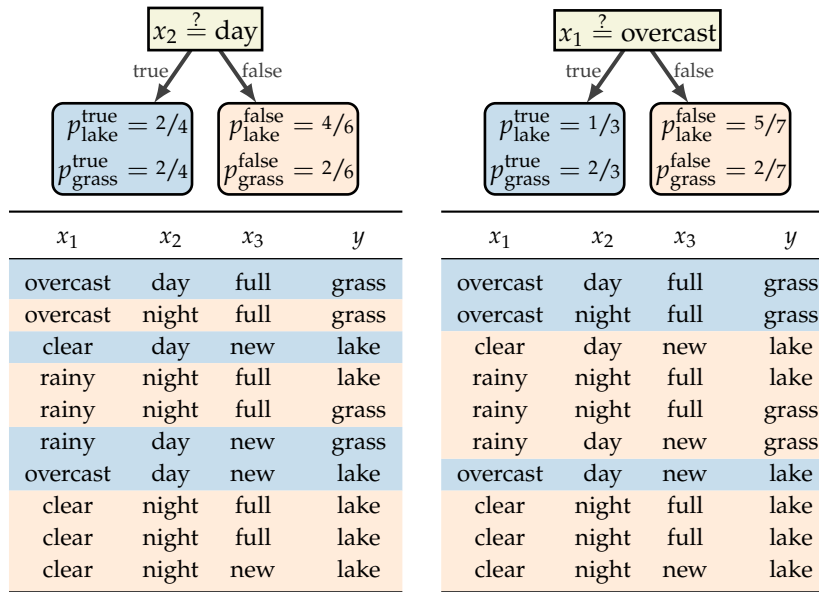


Figure 10.3: Two possible roots of a classification tree and how they split the training data. Each branch shows the fraction of the corresponding training examples with each label.

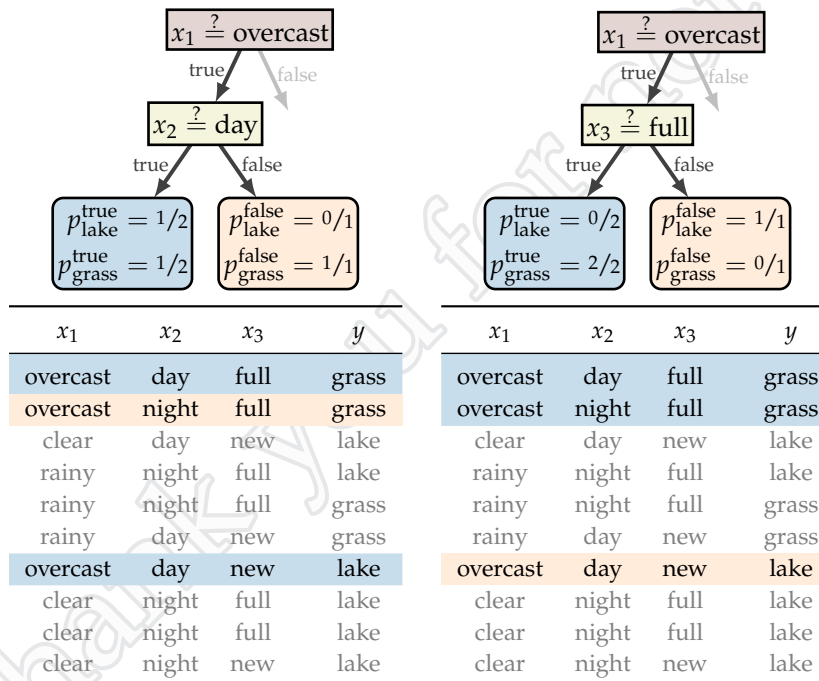


Figure 10.4: Two possible nodes for the “true” branch of the root node, assuming the root was chosen to be $x_1 \stackrel{?}{=} \text{overcast}$. Each branch shows the fraction of the corresponding training examples with each label.

need some way to decide which will lead to a better tree overall, but without actually exploring all possible subtrees.

To do this, we will construct a **scoring function** that evaluates the “goodness” of a particular attribute test. The complete algorithm tries all possible attribute tests and selects the one with the best score. Generally, the scores measure some notion of homogeneity, with a lower score being better. Most commonly, the node score is broken down into an **impurity measure**, G , evaluated for the “true branch” and the “false branch.” If m^{true} is the number of examples that pass to the true branch and m^{false} is the number of examples that pass to the false branch, we would score the potential test as

$$\text{score} = m^{\text{true}} G(\vec{p}^{\text{true}}) + m^{\text{false}} G(\vec{p}^{\text{false}}) \quad (10.2)$$

where $\vec{p}^{\text{true}}$ is the vector of the fraction of examples down the true branch that match each y label (and $\vec{p}^{\text{false}}$ is the same for the false branch). So, we calculate some purity measure for each potential branch and then combine them by weighting each by the number of examples that pass to that branch.

To make this more concrete, consider the possible attribute test in Figure 10.5. In this case, $m^{\text{true}} = 4$ and $m^{\text{false}} = 6$. If we assume the y labels are given in the order lake and then grass, $\vec{p}^{\text{true}} = \begin{bmatrix} 2/4 \\ 2/4 \end{bmatrix}$ and $\vec{p}^{\text{false}} = \begin{bmatrix} 4/6 \\ 2/6 \end{bmatrix}$.

Many impurity measures have been proposed. The most common are misclassification rate, Gini index, and information gain (or entropy). Mathematically, they are

$$G_{\text{MR}}(\vec{p}) = 1 - \max_{c \in \{1, \dots, C\}} p_c \quad \text{misclassification rate} \quad (10.3)$$

$$G_{\text{Gini}}(\vec{p}) = 1 - \sum_{c=1}^C p_c^2 \quad \text{Gini index} \quad (10.4)$$

$$G_{\text{IG}}(\vec{p}) = - \sum_{c=1}^C p_c \lg p_c \quad \text{information gain (or entropy)} \quad (10.5)$$

In each case, c ranges over the possible labels for y . We assume that the possible labels have been given numeric values (like lake = 1 and grass = 2), and C is the total number of labels or classes.

For our example, we have only two labels ($C = 2$). This is a common situation, and one in which the equations above can be slightly simplified to make the calculations easier:

$$G_{\text{MR}}(\vec{p}) = 1 - \max(p_{\text{lake}}, p_{\text{grass}}) = \min(p_{\text{lake}}, p_{\text{grass}}) \quad (10.6)$$

$$G_{\text{Gini}}(\vec{p}) = 1 - p_{\text{lake}}^2 - p_{\text{grass}}^2 = 2p_{\text{lake}}p_{\text{grass}} \quad (10.7)$$

$$G_{\text{IG}}(\vec{p}) = -p_{\text{lake}} \lg p_{\text{lake}} - p_{\text{grass}} \lg p_{\text{grass}} \quad (10.8)$$

Further information

The notation here is abbreviated. The node score and the impurity measure depend on the attribute test being considered, as well as the dataset and which attribute tests have been selected for node above this node on the tree. We are ignoring this extra notation to make the dependencies on the number of examples passing down each branch more explicit.

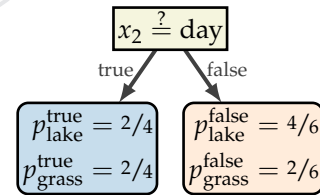


Figure 10.5: Example attribute test to score

Further information

Typically, the logarithm in information gain is taken to be base 2 because that results in a quantity that can be interpreted as bits. However, changing the base will only multiply the score by a constant and not change the attribute test chosen.

The first two equations were simplified by noting that $p_{\text{lake}} = 1 - p_{\text{grass}}$ because there are only two classes.

We can now return to scoring the test in Figure 10.5. If selected the misclassification rate as our score, the total score would be

$$\text{score} = 4 \cdot G_{\text{MR}} \left(\begin{bmatrix} 2/4 \\ 2/4 \end{bmatrix} \right) + 6 \cdot G_{\text{MR}} \left(\begin{bmatrix} 4/6 \\ 2/6 \end{bmatrix} \right) = 4 \cdot \frac{2}{4} + 6 \cdot \frac{2}{6} = 4. \quad (10.9)$$

If we selected the Gini index, the score would be

$$\text{score} = 4G_{\text{Gini}} \left(\begin{bmatrix} 2/4 \\ 2/4 \end{bmatrix} \right) + 6G_{\text{Gini}} \left(\begin{bmatrix} 4/6 \\ 2/6 \end{bmatrix} \right) = 4 \cdot 2 \cdot \frac{2}{4} \cdot \frac{2}{4} + 6 \cdot 2 \cdot \frac{4}{6} \cdot \frac{2}{6} = \frac{14}{3}. \quad (10.10)$$

While this demonstrates the differences and similarities between different impurity measures, a comparison of these two scores is not directly relevant. One particular impurity measure will be selected for the entire learning algorithm. It will be used to calculate the score for each possible attribute test and those scores compared (across different attribute tests) to determine the decision node to add to the tree.

For example, if we were deciding between the two root nodes in Figure 10.3 using the Gini index, we calculate the score for the split resulting from $x_2 \stackrel{?}{=} \text{day}$ and the score for the split resulting from $x_1 \stackrel{?}{=} \text{overcast}$:

$$\begin{aligned} \text{score}_{x_2 \stackrel{?}{=} \text{day}} &= 4G_{\text{Gini}} \left(\begin{bmatrix} 2/4 \\ 2/4 \end{bmatrix} \right) + 6G_{\text{Gini}} \left(\begin{bmatrix} 4/6 \\ 2/6 \end{bmatrix} \right) \\ &= 4 \cdot 2 \cdot \frac{2}{4} \cdot \frac{2}{4} + 6 \cdot 2 \cdot \frac{4}{6} \cdot \frac{2}{6} = \frac{14}{3} \approx 4.67 \end{aligned} \quad (10.11)$$

$$\begin{aligned} \text{score}_{x_1 \stackrel{?}{=} \text{overcast}} &= 3G_{\text{Gini}} \left(\begin{bmatrix} 1/3 \\ 2/3 \end{bmatrix} \right) + 7G_{\text{Gini}} \left(\begin{bmatrix} 5/7 \\ 2/7 \end{bmatrix} \right) \\ &= 3 \cdot 2 \cdot \frac{1}{3} \cdot \frac{2}{3} + 7 \cdot 2 \cdot \frac{5}{7} \cdot \frac{2}{7} = \frac{88}{21} \approx 4.19 \end{aligned} \quad (10.12)$$

Because the score for $x_1 \stackrel{?}{=} \text{overcast}$ is smaller, that attribute test would be preferred. Note that if we had used either of the other two impurity measures, $x_1 \stackrel{?}{=} \text{overcast}$ would also be preferred.

Misclassification rate would seem to be the logical choice for an impurity measure: It corresponds directly to the number of examples incorrectly classified in the training set and so optimizing it minimizes the training error. However, because it is used in a myopic, greedy optimization algorithm, it is not ideal. Our goal is not to pick the attribute test that results in the immediate best gain, but rather the one that results in the best total tree, after both of its branches have been fully grown.

To truly evaluate a particular attribute test would require fully growing each of the subtrees to see the full effects of the choice. This would amount to exploring all possible trees, something we have

Further information

For comparison, using G_{IG} ,

$$\text{score}_{x_2 \stackrel{?}{=} \text{day}} \approx 9.51$$

$$\text{score}_{x_1 \stackrel{?}{=} \text{overcast}} \approx 8.797$$

and using G_{MR} ,

$$\text{score}_{x_2 \stackrel{?}{=} \text{day}} = 4$$

$$\text{score}_{x_1 \stackrel{?}{=} \text{overcast}} = 3$$

already established as prohibitive due to the doubly-exponentially large number of possible trees. Therefore, we need to view the scoring function as a heuristic approximation of the total effect of growing a full tree starting with a particular attribute test.

Figure 10.6 compares these three impurity measures for the case of two classes. Only the fraction of examples of one class is necessary to plot because the other fraction must be one minus this amount. Consider the case where we have a split that results in $p_{\text{lake}} = 0.3$ ($p_{\text{grass}} = 0.7$) down the true branch and $p_{\text{lake}} = 0.7$ ($p_{\text{grass}} = 0.3$) down the false branch (a very symmetrically mediocre split). That is, mostly the examples where Sibonelo is found in the grass go down the true branch and mostly the examples where Sibonelo is found at the lake go down the false branch.

There may be another attribute test that results in both branches improving (say one where the true branch has $p_{\text{lake}} = 0.2$ and the false branch has $p_{\text{grass}} = 0.9$). In this case, all of the impurity scores would prefer this new attribute test. This is not a difficult comparison to make.

The more difficult choice to evaluate is one where one branch is better, but the other is worse. For instance, if the true branch has $p_{\text{lake}} = 0.2$, but the false branch has $p_{\text{lake}} = 0.6$. Whether the original attribute test with $\vec{p}^{\text{true}} = \begin{bmatrix} 0.3 \\ 0.7 \end{bmatrix}$, $\vec{p}^{\text{false}} = \begin{bmatrix} 0.7 \\ 0.3 \end{bmatrix}$, or the new one with $\vec{p}^{\text{true}} = \begin{bmatrix} 0.2 \\ 0.8 \end{bmatrix}$, $\vec{p}^{\text{false}} = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}$ would be preferred depends on how many examples go down each branch in each case. For simplicity, assume that both attribute tests send half of the examples down each branch (which means that the total score is proportional to the sum of the branch scores).

The misclassification rate score is indifferent between these two options: both get 30% of the examples wrong, it is just a question of down which branch. By contrast, both Gini scoring and information gain prefer the second option. Look at Figure 10.6. G_{Gini} and G_{IG} have larger slopes near the edges (more pure) sides of the plot. Thus, moving p_{lake} from 0.3 to 0.4 increases the score less than moving it from 0.3 to 0.2 decreases the score. Empirically, this preference to be even more pure down one branch (at the expense of the other branch) leads to smaller, better trees.

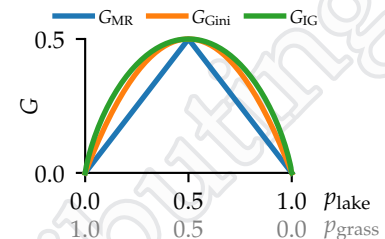


Figure 10.6: Comparison of three impurity scores for binary classification. G_{IG} has been scaled by $1/2$ to match the height of the other scores.

Further information

There are also theoretic arguments that can be made to justify either the Gini score or information gain.

Example

We will put these pieces together and learn a decision tree for the task of finding Sibonelo. We will choose to use the Gini index as our impurity score. Our algorithm begins by finding the best attribute test for the root node. There are seven possible attribute tests. They are shown in Figure 10.7 (Figure 10.3 only showed two of them).

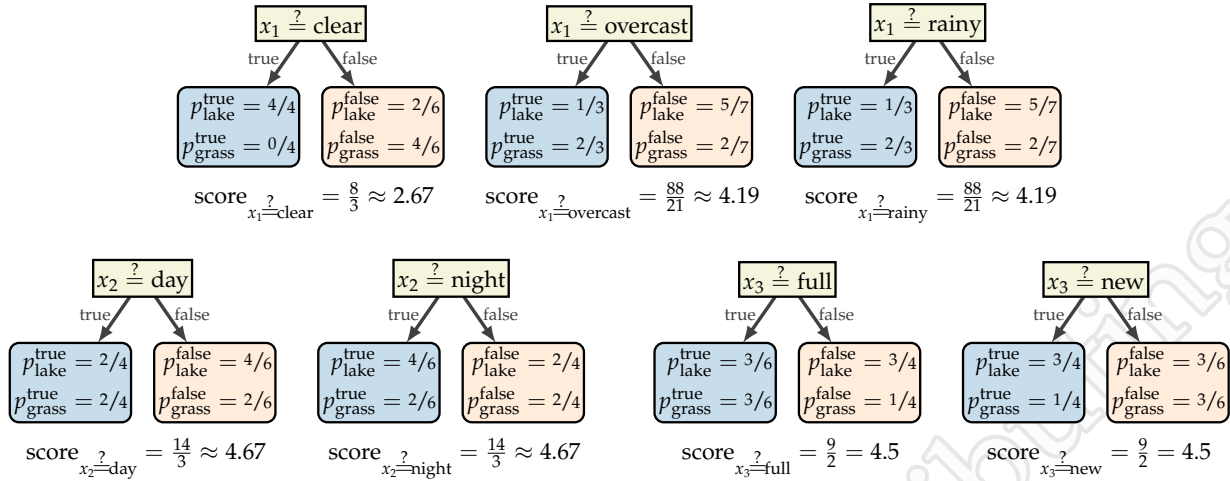


Figure 10.7: Gini scoring for all possible root nodes

Before continuing on with the example, note that for binary features (like x_2 and x_3 here), the two possible attribute tests are symmetric. Therefore, there is no need to check both (although it would not hurt, other than to take slightly more computation time). For features with more values (like x_1 here), equality tests against all feature values are necessary because they all produce different splits of the data.

The learning algorithm selects the one with the lowest score: in this case, the test $x_1 \stackrel{?}{=} \text{clear}$. This happens to be the first one checked, but all possible test must be checked. Only if the score returns exactly 0 (an indication that the test perfectly separates the classes) can we guarantee that no other options will be better.

Once this root node is fixed, the algorithm proceeds recursively down both branches. Arbitrarily taking the “true” branch first, we consider only those training examples for which $x_1 = \text{clear}$. All of these examples have the same label ($y = \text{lake}$), so this branch is easy: just a leaf of “lake.”

Continuing down the “false” branch of the root node, all of the attribute tests will be tried again. Of the seven, only three are interesting: Selecting $x_1 \stackrel{?}{=} \text{clear}$ again will not split the data; we need only pick either one of the two tests for x_2 or x_3 (the binary features), as mentioned before; and because x_1 is now essentially a binary feature (only two values are left down this branch of the tree), $x_1 \stackrel{?}{=} \text{overcast}$ and $x_1 \stackrel{?}{=} \text{rainy}$ will have the same effective (symmetric) split. We show the three splits worth considering in Figure 10.8.

The subtree with $x_3 \stackrel{?}{=} \text{full}$ has the lowest score, so we fix this and continue. Down its “true” branch, there are still four remaining training examples and two attribute tests worth trying: $x_1 \stackrel{?}{=} \text{overcast}$

Further information

While we have omitted checking some attribute tests because we know they are redundant, in practice it is often simpler for the computer to just try them all.

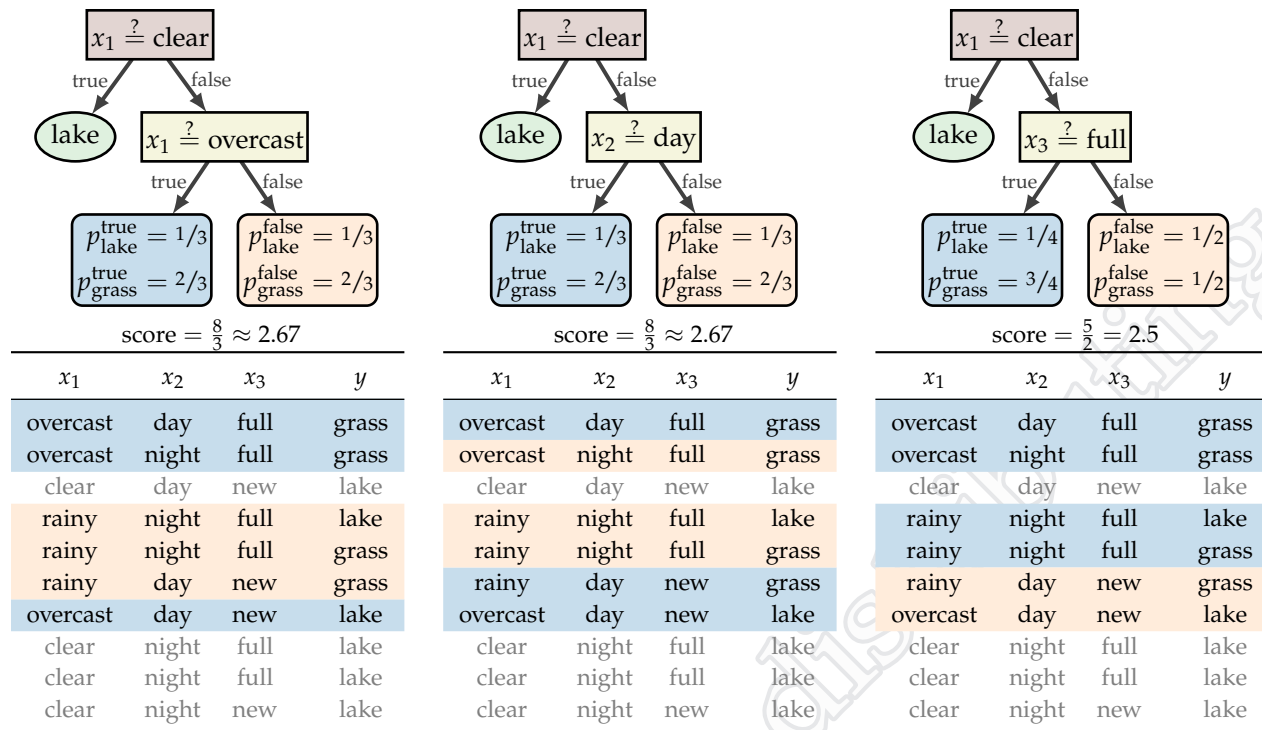


Figure 10.8: Gini scoring for the second test in learning an example decision tree.

(or equivalently $x_1 = \text{rainy}$) and $x_2 = \text{day}$ (or equivalently $x_2 = \text{night}$). Gini scoring would prefer $x_1 = \text{overcast}$ (by a score of 1.00 to 1.33). This results in the tree shown in Figure 10.9. The “true” branch here corresponds to $x_1 \neq \text{clear}$ and $x_3 = \text{full}$ and $x_1 = \text{overcast}$. There are only two examples of an overcast day with a full moon (the first and last examples). In both, Sibonelo was in the grass ($y = \text{grass}$, so we do not continue and just use a leaf with this value.

The “false” branch ($x_1 \neq \text{clear}$ and $x_3 = \text{full}$ and $x_1 \neq \text{overcast}$) corresponds to examples where it is rainy and there is a full moon. There are only two of these examples. They differ in their y values (in one case, Sibonelo was in the lake and in the other Sibonelo was in the grass). However, their $\vec{x}$ values are the same, so there is no ability to distinguish between them with an attribute test. So, we do not continue here either and use a leaf with the majority class. For this particular example both classes are equally frequent in the remaining example, so we just pick one arbitrarily.

Finally, for the “?” node in Figure 10.9, there are only two examples which differ only in x_1 , so replacing the “?” with $x_1 = \text{overcast}$ differentiates them, and the resulting final learned tree is shown. The general learning algorithm is shown in Algorithm 10.10.

Key point

Growing stops if all examples that get to that node have the same y values.

Key point

Growing stops if all examples that get to that node have the same $\vec{x}$ values.

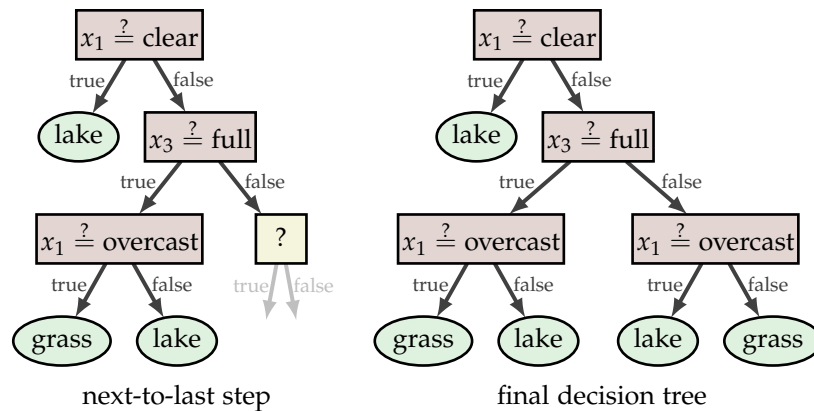


Figure 10.9: The next-to-last and last step in growing a decision tree

Continuous Features

If there are features that are real-valued, we consider attribute tests that check for *inequality* instead of equality. For instance, if we were to add a fourth feature to the Sibonelo spotting example that is the atmospheric pressure (measured in the number of hectopascals, hPa, greater than 1000), we cannot have attribute tests like $x_4 \stackrel{?}{=} 10.3$ because such an exact value of 10.3 might never exactly appear again in the data. Rather, we consider attribute tests like $x_4 \stackrel{?}{\leq} 10.3$. In this case, 10.3 is the **threshold**. Examples with atmospheric pressures less than the threshold go down the true branch. Those with larger atmospheric pressures go down the false branch.

For discrete features, there are a (generally small) finite number of attribute tests, because there only a few values the feature can be compared to. But, for continuous (real-valued) features, there are an infinite number of possible thresholds. This would seem to be a problem; not even a fast computer has time to check all infinite number of them!

Consider the training data in Table 10.11. While the attribute test $x_4 \stackrel{?}{\leq} 7.9$ and the attribute test $x_4 \stackrel{?}{\leq} 8.2$ are different and may well produce different results on future testing data, they separate the *training data* the same way: there are no training examples with values for x_4 between 7.9 and 8.2. In fact, any threshold between 7.5 and 8.7 is equivalent in terms of the training set. The training data gives us no information on which to prefer thresholds that lie between the values observed in the training data.

Thus, there are only a maximum of $m - 1$ effectively different thresholds for a particular training set of m examples.¹ We only need to consider one threshold between each consecutive feature value.

¹ There is no need to consider thresholds outside of the range of training values.

DTLEARN

In: X, Y **Out:** $\mathcal{T}$

if all rows of X are the same or all elements of Y are the same **then**
 $y \leftarrow \text{PLURALITY}(Y)$
return leaf(y)
else
 $SC_{\text{best}} \leftarrow \infty$
for all possible tests of single attributes, r **do**
 $(X^{\text{true}}, Y^{\text{true}}), (X^{\text{false}}, Y^{\text{false}}) \leftarrow \text{SPLIT}(r, X, Y)$
 $m^{\text{true}} \leftarrow \text{LENGTH}(Y^{\text{true}})$
 $m^{\text{false}} \leftarrow \text{LENGTH}(Y^{\text{false}})$
 $SC \leftarrow m^{\text{true}} \text{SCORE}(Y^{\text{true}}) + m^{\text{false}} \text{SCORE}(Y^{\text{false}})$
if $SC < SC_{\text{best}}$ **then**
 $SC_{\text{best}} \leftarrow SC$
 $r_{\text{best}} \leftarrow r$
 $(X^{\text{true}}, Y^{\text{true}}), (X^{\text{false}}, Y^{\text{false}}) \leftarrow \text{SPLIT}(r_{\text{best}}, X, Y)$
 $\mathcal{T}^{\text{true}} \leftarrow \text{DTLEARN}(X^{\text{true}}, Y^{\text{true}})$
 $\mathcal{T}^{\text{false}} \leftarrow \text{DTLEARN}(X^{\text{false}}, Y^{\text{false}})$
return node($r_{\text{best}}, \mathcal{T}^{\text{true}}, \mathcal{T}^{\text{false}}$)

Algorithm 10.10: Decision Tree Learning Algorithm.

PLURALITY returns the most common element of its input vector.

LENGTH returns the number of elements in its input vector.

SCORE is the scoring function chosen for learning.

SPLIT takes an attribute test and a dataset and splits the dataset according to the test.

weather	time	moon	pressure	location
overcast	day	full	3.4	grass
overcast	night	full	4.1	grass
clear	day	new	6.1	lake
rainy	night	full	7.5	lake
rainy	night	full	8.7	grass
rainy	day	new	9.1	grass
overcast	day	new	9.1	lake
clear	night	full	10.3	lake
clear	night	full	10.4	lake
clear	night	new	11.4	lake

Table 10.11: Extended Sibonelo-spotting data, conveniently sorted by x_4 , the atmospheric pressure.

For x_4 (pressure) in Table 10.11, that would be one threshold between 3.4 and 4.1 and one threshold between 4.1 and 6.1, and so on. Which exact threshold? We are free to choose, but it is common to select the value exactly half-way between the two consecutive data values. So, for x_4 for Table 10.11, we would consider thresholds of 3.75, 5.1, 6.8, 8.1, 8.9, 9.7, 10.35, and 10.9. Note, if two consecutive values are the same (9.1 in this example), we cannot produce a threshold “between” them. Figure 10.12 shows the classification tree learned on this data using Gini scoring.

Computational Complexity

For a discrete feature with d possible values, we need to calculate the score d times (once for each value that the attribute could be compared to). For each of these, we can calculate $\vec{p}^{\text{true}}$ and $\vec{p}^{\text{false}}$ with a single pass over all of the data. At the root node, there are m training examples and thus to consider all possible tests for this feature would take $O(dm)$ time.

For a continuous feature, there are $m - 1$ possible values. And, we can score each of them with a single pass over the training data. So, this would result in taking $O(m^2)$ time. However, we can do better.

To find the $m - 1$ thresholds, we can sort the data according to the feature value in question. This takes $O(m \ln m)$ time. But, we can score all of the possible thresholds with a single extra pass over the data: We keep track of the number of examples of each class on the true/less-than branch and the number of examples of each class on the false/greater-than branch. Starting with the smallest threshold, all but one of the examples are on the greater-than branch. Then, as we scan the threshold to larger split points, one-by-one each example moves from the greater-than branch to the smaller-than branch, requiring only decrementing a count from one side and incrementing the corresponding count from the other side. At any point, these counts can be turned into the empirical probability vectors necessary for scoring by dividing by their sums. This results in a total time for evaluating all thresholds for a continuous feature of $O(m \ln m)$.

Therefore, if there are n_d discrete features, each with d possible values and n_c continuous features, the total time to find the best-scoring root node is $O(n_d dm + n_c m \ln m)$.

Combating Overfitting

A decision tree grown in the manner above will almost certainly overfit. Unless two examples have the same $\vec{x}$ but different y values,

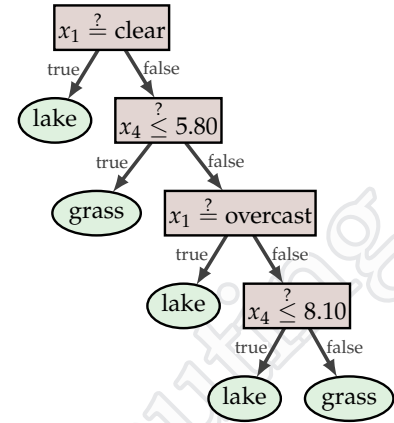


Figure 10.12: Classification tree learned on the extended Sibonelo-spotting data of Table 10.11.

all training examples will be correctly classified. For most problems, 100% training accuracy is a sure sign of overfitting.

A natural way of trying to control overfitting would be to stop growing the tree before necessarily reaching pure leaves. There are two common ways of doing this: stopping growing at a depth threshold and stopping growing when the number of training examples per leaf is lower than a threshold. Either or both can be effective. However, it is hard to set a single fixed threshold (for either depth or number of examples) that works well across the whole tree.

For this reason, more commonly an entire tree is grown and *then* it is pruned back based on a pruning criterion. **Decision tree pruning** replaces entire subtrees with a single leaf. Instead of stopping early (which may miss the advantage of continuing past the immediate next level in the tree), pruning allows us to see the full advantage of growing an entire subtree before deciding whether to stop the tree's growth at that point.

Pruning works recursively backward from the leafs to the root, at each node deciding whether to replace the subtree rooted at the node with a leaf. Because it works backward up the tree, nodes closer to the root make their decision of whether to prune based on already pruned subtrees. We need to make two choices to complete the definition of this algorithm: how to decide what leaf to use if pruned and how to decide whether to prune.

If a subtree is to be pruned, it is replaced with a single leaf. We select the leaf that would have been picked *if* the growing algorithm had stopped at this point. More specifically, the leaf reports the plurality² class of the training examples that fell into this node. Therefore, during learning we must keep track for each internal node of which y -value would be chosen if the tree had stopped growing at that node. Figure 10.13 shows this for our original example (without the atmospheric pressure).

² fancy word for "most common"

Pruning amounts to selecting which subpart of the grown tree to keep. For the example in Figure 10.13, there are six possible decision trees that could result from pruning (ranging from a tree of a single leaf to the full tree without any pruning). We cannot select among them based on training error, because the tree has been grown to the point where it has minimal training error. Just based on the training data, we would choose to do no pruning. (It could only increase the training error.)

Cost-Complexity Pruning

One approach is to add a regularizer term to the training loss and prune to minimize this new loss function. **Cost-complexity pruning**

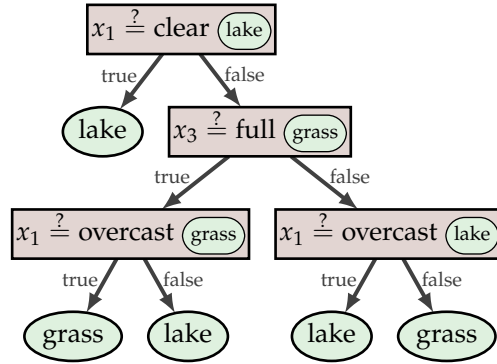


Figure 10.13: The learned decision tree, with the leaves that would be used if a node is to be pruned.

is the most common form, where the regularizer is just the number of nodes in the tree. Thus, the loss used for pruning is

$$L_{CC} = \sum_{i=1}^m l(y_i, f_{\mathcal{T}}(\vec{x}_i)) + \lambda \|\mathcal{T}\|. \quad (10.13)$$

If $\mathcal{T}$ represents the tree, $f_{\mathcal{T}}$ is the function associated with the tree, and $\|\mathcal{T}\|$ is the size of the tree measured as the number of decision nodes. The per-item loss $l(y, f)$ for pruning a classification tree is chosen to be the zero-one loss: 0 if the prediction is correct and 1 if the prediction is incorrect.

Given a regularization strength, λ , we grow the tree as normal (almost certainly overfitting), and then prune back to make L_{CC} as small as possible. Equation 10.13 penalizes according to the sum of the number of misclassified (training) examples plus λ times the number of nodes. Therefore, when considering whether to prune a particular node, we need only consider the number of nodes in the subtree rooted at this node and the number of misclassifications that can be attributed to this subtree. Each node saved is “worth” (or “makes up for”) λ misclassifications in the loss function. Therefore, if the number of misclassifications added by pruning the subtree is greater than λ times the number of nodes in the subtree, the subtree is pruned (replaced by a leaf).

Figure 10.14 shows the number of errors each leaf (and node, if the tree is pruned to at that node) makes on the training set (as a fraction of the total number of examples that make it to that leaf). If we pick $\lambda = 1/3$, the pruning proceeds as follows.

- Starting at the bottom, we consider pruning the node labeled **G**. If pruned, it would make 1 error for a total cost of $1 + \lambda \cdot 0 = 1$. In total its children make 0 errors, for a total cost of $0 + \lambda \cdot 1 = 1/3$. Since pruning costs more, we keep this node.

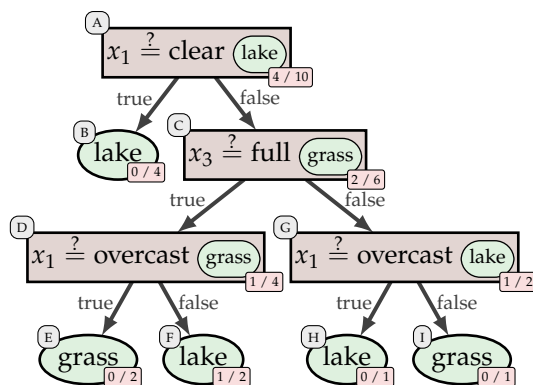


Figure 10.14: The learned decision tree, with the number of misclassifications made on the training data (lower right corners). For internal decision nodes, this number is *if* the node were replaced with a leaf. Nodes and leaves are also marked (upper left corners) for reference.

- Proceeding to node **D**, pruning it would make 1 error and in total its children make 1 error. Therefore, we prune it because $1 + \lambda \cdot 0 < 1 + \lambda \cdot 1$.
- For node **C** (proceeding back up the tree), if we prune we make 2 errors for a cost of $2 + \lambda \cdot 0 = 2$. If we keep the subtree, it now has two nodes (C and G, because the true branch has been pruned) and it makes 1 error³, for a cost of $1 + \lambda \cdot 2 = 5/3$. Keeping the subtree is less costly, so we do not prune.
- Finally, for node **A**, if we prune the cost is $4 + \lambda \cdot 0 = 4$. If we keep the tree, we make 0 errors down the true branch and 1 error down the false branch (the error is made at the newly pruned node **D**). There are 3 total nodes, so the score is $1 + \lambda \cdot 3 = 2$. Because this is less than the pruning cost, we do not prune at the root.

The final tree is shown in Figure 10.15.

Different regularization strengths, λ , will result in different trees. A larger λ will result in smaller trees because the increased regularizer value from the number of nodes must be outweighed by the reduction in the training error by keeping the larger tree. For instance, if we had instead picked $\lambda = 3/2$, node **B** would also have been pruned, leaving only the root as a decision node.

The total learning algorithm is the combination of growing and pruning the tree on the training set. This learning method has a hyper-parameter λ which is usually selected with hold-out validation or cross-validation.

Note that λ does not play a role in the growing of the tree (only its pruning). Therefore, in hold-out validation, we only need to grow the tree once. For each λ value, we prune the tree (again, using the training set). We then score this value of λ based on the error (with-

³ This error is at the pruned node **D**. The subtree at **G** does not make any errors because it was not pruned and its leaves do not make any errors.

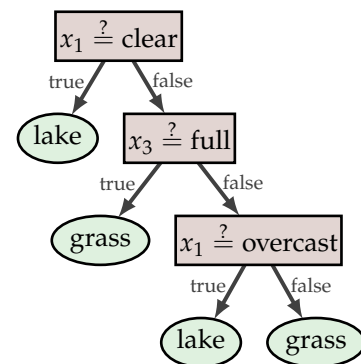


Figure 10.15: Decision tree pruned using cost-complexity with $\lambda = 1/3$.

weather	time	moon	location					
clear	night	full	lake	}	B	}	A	
clear	night	new	lake					
overcast	night	full	lake	}	E			D
overcast	night	full	grass					
rainy	day	full	lake	}	F			
overcast	day	new	grass					}
overcast	day	new	grass					
rainy	day	new	lake	}	G			
rainy	day	new	grass					}
rainy	night	new	grass					

Table 10.16: Sibonelo-spotting validation (pruning) data. Noted on the right are the nodes or leaves in Figure 10.17 into which each hippopotamus (example) falls.

out regularization) the resulting tree yields on the validation set. We select the λ value with the best (lowest) score.

Reduced Error Pruning

In cost-complexity pruning, different λ values result in different pruned trees. We select among these based on a validation set. However, we could omit λ altogether and just prune directly based on a validation set.

This method is called **reduced error pruning**. We build the tree based on the (parameter) training set. We then prune based on whether the error on a (separate) validation set is better or worse.

As an example, consider the *validation data* in Table 10.16. This data was not used in training and instead held out for pruning. It has been (conveniently) sorted according to how each hippopotamus falls⁴ into the leaves of the learned tree.

Figure 10.17 shows the learned tree with the pruning set errors. Note that this differs from Figure 10.14 which shows the *training* set errors. In cost-complexity pruning, we needed a regularization penalty to help mitigate that our pruning was based on the same data as training, and therefore we are likely overly optimistic about the quality of our tree (pruned or unpruned). By contrast, in reduced error pruning, we use an independent validation set for pruning. Therefore, we generally do not use regularization when doing reduced error pruning. Instead, we prune just based on whether it improves the number of errors on the pruning set.

Reduced error pruning proceeds in the same bottom-up fashion as

Further information

When a validation set is used for pruning it is sometimes called a **pruning set** instead.

⁴ ker-splat

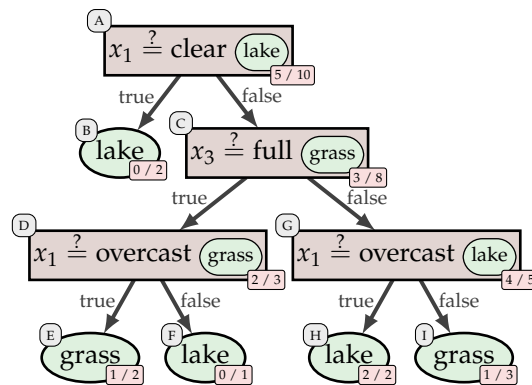


Figure 10.17: The learned decision tree, with the number of misclassifications made on the pruning data (lower right corners). For internal decision nodes, this number is *if* the node were replaced with a leaf. Nodes and leaves are also marked (upper left corners) for reference.

cost-complexity pruning:

- Starting with node **G**, its children make 3 total errors, but it makes 4 errors on the pruning set. Therefore, we do not prune.
- Proceeding to node **D**, its children make 1 error, but it makes 2 errors. Therefore, we do not prune.
- For node **C**, its subtrees make a total of 4 errors (1 at **E**, 0 at **F**, 2 at **H**, and 1 at **I**; recall we have *not* pruned **D** or **G**). However, if we prune at **C**, the resulting leaf makes only 3 errors on the pruning set. Therefore, we do prune.
- Finally, for node **A**, its leaves make 3 errors (0 at **B** and 3 at **C**), but it makes 5 errors on the pruning set. Therefore we do not prune.

The final tree is shown in Figure 10.18.

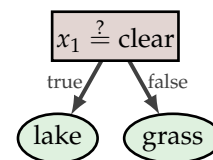


Figure 10.18: Reduced error pruned decision tree

Regression Trees

Although not the focus of this chapter, decision trees can also be used for regression, in which case they are also called **regression trees**. For regression, the leaves still report a constant y -value and the nodes still make decisions based on a single attribute. If the attributes are continuous, this means that the learned function is a piece-wise constant function of the input. (See Figure 10.20 as an example.)

Regression trees are learned in the same fashion as classification trees. However, we need a different impurity measure. Instead of the Gini index or the information gain, mostly commonly we used the average squared error. That means that, to score a branch that has

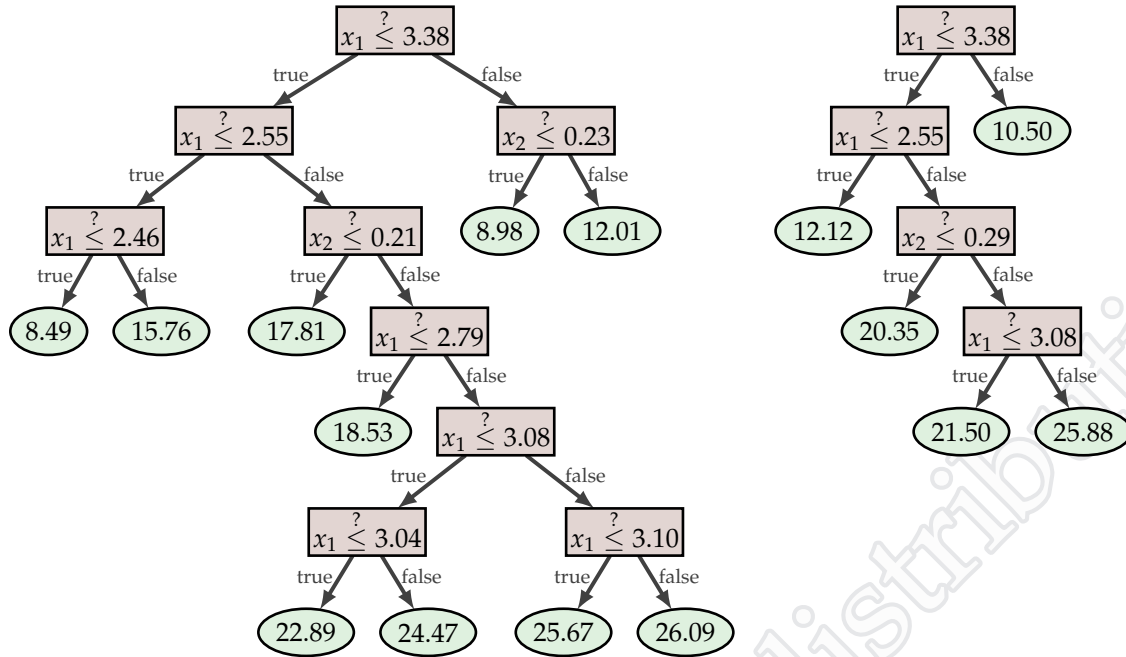


Figure 10.19: Example of regression trees learned on the data from Table 2.1. On the left is the full tree. On the right is the tree when restricted to make sure at least two training examples fall into each leaf.

examples with targets $y_1, \dots, y_{m'}$, we use the impurity measure

$$G_{SE}(y_1, \dots, y_{m'}) = \frac{1}{m'} \sum_{i=1}^{m'} (y_i - \mu)^2 \quad (10.14)$$

where

$$\mu = \frac{1}{m'} \sum_{i=1}^{m'} y_i \quad (10.15)$$

which measures the average squared error between the training value and the (constant) prediction. Figure 10.19 shows two learned regression trees.

Variants

The decision trees above all use binary decisions based on a single feature at the internal nodes. We could consider having non-binary splits. For instance, instead of asking if the weather is overcast, we could just have one branch for each of the (three) possible weather types. This is seldom done for two reasons: First, such multi-way decisions can be constructed using multiple layers of binary decisions, and therefore they are not necessary to be able to represent any particular function. Second, such multi-way splits cause the

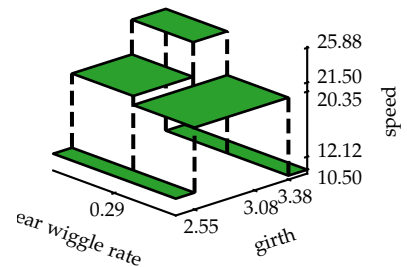


Figure 10.20: Plot of the function represented by the regression tree shown on the right in Figure 10.19.

data to fragment more quickly, which leads to more overfitting with less chance of pruning (because there are fewer levels). It is better to aggregate data together where possible. For instance, it may be that overcast and rainy days are more similar and should be treated roughly equally. If we split them apart anytime we have a decision node based on weather, we lose that ability.

For features with many possible values, it might make sense not to check for equality to a specific value, but rather to check whether the feature's value falls into a specific set. This does not help with features that have two or three values. But, consider a feature like "primary ingredient" for predicting whether a hippopotamus would like to eat a particular dish. If the possible values for the ingredient are alfalfa, beef, chicken, corn, grass, mutton, and squash. It might make sense to be able to produce a decision like, "Is the primary ingredient one of alfalfa, corn, grass, or squash?"⁵ But, we do not want to have to form such subsets ahead of time, but instead learn them. If a feature can take v values, there are 2^{v-1} possible subsets to try. With so many possible decisions, one might look good purely by coincidence, leading to overfitting. There are methods that try to search heuristically over such subset relationships without explicitly checking all 2^{v-1} possible decision nodes. They are not covered here.

⁵ Hippopotamuses are primarily herbivores.

Particularly for continuous-valued features, it might make sense to make a decision based on the value of multiple features. For instance, we could check whether a linear combination of two (or more) features exceeds a threshold. Searching for the best possible combination can be challenging. It also takes the decision tree away from its simplicity. If such more complex decision points are necessary, other methods are probably more appropriate.

Relative Merits

Decision trees are different from previous methods. They have distinct advantages and disadvantages that are worth keeping in mind.

Disadvantages

Decision tree learning is unstable. This does not mean that they will literally fall over, but rather that small changes in the input (training set) can lead to large changes in the output (learned decision tree).

This can be handled with more complex ways of using decision trees, but such methods lose many of the advantages of decision trees.

Decision trees, particularly regression trees, cannot produce linear functions. Or, more generally, they cannot produce smooth or continuous functions. This is inherent in the idea of making splits, where

one part of the data is then treated as being completely different from the other part. While this can be an advantage in some domains, in many it is seen as a disadvantage as it disallows even the simplest continuous functions: linear functions.

Advantages

Decision trees can be **interpretable**. That is, a human can look at the function learned and understand how it is forming its decisions. As a straight-forward flowchart, a decision tree is often held up as an interpretable machine learning method, particularly when contrasted with a neural network that makes a series of complex vector calculations for which it is difficult to attribute “meaning.”

However, this interpretability should be taken cautiously. On real-world problems, the tree is often very large, even after pruning. Inspection of the tree may not be possible. Further, while it is tempting to inspect the root node (or other splits near the top of the tree) and consider them as the primary drivers of the resulting function, this can be misleading. As mentioned above, decision trees can be unstable and thus the decision that sits at the top (or near the top) can change drastically even if the dataset is only slightly modified. Thus, it may not be a good indication of what was actually learned as important from the training data.

Decision trees can handle **missing values**. The other methods we have considered depend on having all features be measured for all training and testing examples. In many applications this is not realistic. A hippopotamus will not have every possible medical measurement taken each time she visits the veterinarian. Decision trees can handle missing values by providing a third branch (unknown) beside the true and false branches for any decision node. There are different ways to learn and incorporate the extra branch into the tree (which are also beyond our scope), but they are standard in many decision tree learning methods.

Decision trees naturally handle categorical and continuous-valued features together. This is less of an advantage, compared with those above, because other methods can be modified to handle both as well. However, it is particularly straight-forward with decision trees.

Finally, decision trees are computationally fast to learn. They share this in common with naive Bayes and linear regression. They do not require the lengthy iterations of gradient descent that neural networks (and to a lesser degree logistic regression) require. A few scans of the data are usually sufficient, and at each level of the learned tree, the number of examples per node decreases, making the learning algorithm very fast, even on large datasets.

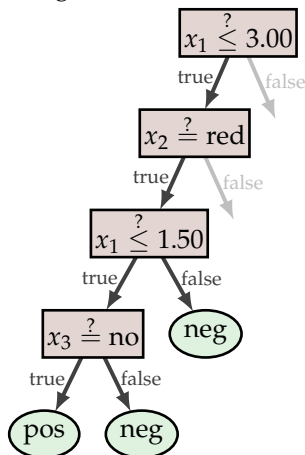
Exercises

Exercise 10.1

Construct (no pruning) a decision tree from the training data in Table 10.21. Use the Gini score to select the decision nodes. Break ties as you see fit. Treat x_1 as real-valued and x_2 as categorical.

Exercise 10.2

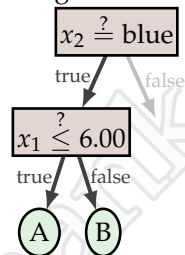
Below is a decision tree in the middle of being learned from the training set in Table 10.22.



- Using the Gini index, what decision node should be added next? Show your calculations.
- What is the resulting decision tree if grown to completion?
- Prune using cost-complexity pruning, with $\lambda = 0.75$.

Exercise 10.3

Below is a decision tree in the middle of being learned from the training set in Table 10.23.



Using the Gini index, what decision node should be added next? Show your calculations.

Exercise 10.4

Below is decision tree that was learned from data (not shown). The labels circled inside the decision nodes indicate what the learning

x_1	x_2	y
1	1	B
1	0	B
1	0	B
3	0	A
3	0	A
3	0	B
5	0	B
5	1	A
5	1	B

Table 10.21: Toy decision tree training dataset

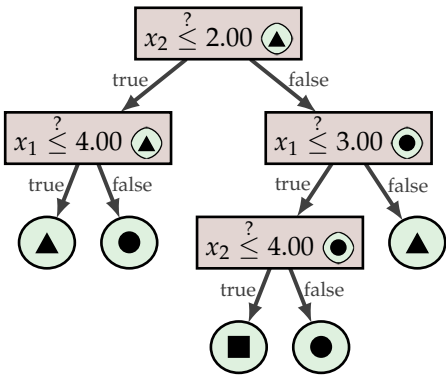
x_1	x_2	x_3	y
1	red	no	pos
1	red	yes	neg
2	red	yes	neg
2	red	no	neg
1	blue	yes	neg
1	blue	yes	pos
1	green	no	neg
2	blue	yes	pos
2	green	yes	pos
4	red	no	pos
4	red	no	pos
5	blue	no	pos
5	green	yes	pos

Table 10.22: Another toy decision tree training dataset

x_1	x_2	y
1	red	C
3	red	C
7	red	A
7	red	B
1	green	B
3	green	C
7	green	C
7	green	A
1	blue	A
1	blue	A
3	blue	A
5	blue	A
7	blue	B

Table 10.23: Yet another toy decision tree training dataset

algorithm would have selected as the label if the tree had not been grown further.



Prune the decision tree using reduced error pruning. Assume on a tie, that the subtree is pruned. For each part below, start with the tree above and use a different pruning set.

- a. Use the pruning set in Table 10.24.
- b. Use the pruning set in Table 10.25.

x_1	x_2	y
3	1	▲
2	0	■
5	1	▲
5	1	●
1	3	●
1	5	●
1	5	■
4	4	▲
4	5	▲

Table 10.24: Toy decision tree pruning dataset

Exercise 10.5

Grow (learn) a regression tree for the data in Table 10.26

Exercise 10.6

Consider a dataset with n binary features and m examples. In big-O notation, what is the running time of learning a decision tree from this dataset if the tree is grown to a maximum depth of d ?

x_1	x_2	y
1	0	▲
4	0	●
6	0	▲
1	3	■
1	3	■
1	5	●
4	5	▲
1	6	■
4	6	●
4	7	●
6	7	●

Table 10.25: Another toy decision tree pruning dataset

x_1	x_2	y
0	0	4
0	2	1
2	0	4
2	0	2
2	3	2

Table 10.26: Regression tree training dataset

